



Chapter: Strategically Automating SQL

Topic: Triggers in SQL

Section 1: Learn

1.1 What Are Triggers in SQL?

A **trigger** in SQL is a **special program block** that runs **automatically** in response to specific events on a table or view.

These events include **INSERT**, **UPDATE**, and **DELETE**. Triggers help enforce rules, maintain audit logs, or automate repetitive actions without relying on applications or user intervention.

1.2 Why Use Triggers?

- **Data Integrity:** Enforce business rules automatically.
 - **Auditing:** Keep logs of changes made to sensitive data.
 - **Automation:** Perform calculations or checks without needing manual intervention.
 - **Cascading Updates:** Maintain consistency across related tables.
 - **Validation:** Block transactions that don't meet certain conditions.
-

1.3 Structure of a Trigger

Every trigger must define:

- **WHEN** it should fire (BEFORE or AFTER the event)
- **WHAT** event causes it (INSERT, UPDATE, DELETE)
- **WHERE** it applies (which table)



- **WHAT** logic it should execute
-

1.4 Trigger Timing Options

- **BEFORE Triggers:** Execute **before** the action happens.
 - Use when you want to validate or change the data being inserted/updated.
 - **AFTER Triggers:** Execute **after** the action completes.
 - Use when you want to log, calculate, or propagate data elsewhere.
-

1.5 Trigger Event Types

You can define triggers on the following actions:

- **INSERT** – fires when new rows are inserted.
- **UPDATE** – fires when rows are modified.
- **DELETE** – fires when rows are removed.

You can combine these with timing:

- BEFORE INSERT
- AFTER UPDATE
- BEFORE DELETE

...and so on.

1.6 Syntax to Create a Trigger (MySQL Style)

```
CREATE TRIGGER trigger_name
[BEFORE | AFTER] [INSERT | UPDATE | DELETE]
ON table_name
FOR EACH ROW
```



```
BEGIN  
    -- Trigger logic here  
END;
```

1.7 Example 1: Audit Log on Delete

Log employee deletions automatically:

```
CREATE TRIGGER audit_employee_delete  
AFTER DELETE ON employees  
FOR EACH ROW  
BEGIN  
    INSERT INTO employee_audit (emp_id, name, deleted_at)  
    VALUES (OLD.id, OLD.name, NOW());  
END;
```

1.8 Example 2: Prevent Low Salary Entries

Stop entries where salary is less than ₹10,000:

```
CREATE TRIGGER salary_guard  
BEFORE INSERT ON employees  
FOR EACH ROW  
BEGIN  
    IF NEW.salary < 10000 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Salary below company minimum!';  
    END IF;
```



```
END;
```

1.9 Using OLD and NEW in Triggers

- **OLD.column_name** – refers to the value **before** the action (used in UPDATE and DELETE).
- **NEW.column_name** – refers to the value **being inserted or updated**.

Example in an UPDATE trigger:

```
CREATE TRIGGER log_salary_change  
AFTER UPDATE ON employees  
FOR EACH ROW  
BEGIN  
    INSERT INTO salary_changes (emp_id, old_salary, new_salary, changed_on)  
    VALUES (OLD.id, OLD.salary, NEW.salary, NOW());  
END;
```

1.10 Deleting a Trigger

```
DROP TRIGGER IF EXISTS trigger_name;
```

Section 2: Practise

Exercise 1: Block Deletion of VIP Customers

```
CREATE TRIGGER block_vip_delete  
BEFORE DELETE ON customers
```



```
FOR EACH ROW
BEGIN
    IF OLD.customer_type = 'VIP' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete VIP customer!';
    END IF;
END;
```

Exercise 2: Create Audit Trail for Student Records

```
CREATE TRIGGER student_audit_log
AFTER INSERT ON students
FOR EACH ROW
BEGIN
    INSERT INTO student_log (student_id, name, action, timestamp)
    VALUES (NEW.id, NEW.name, 'INSERT', NOW());
END;
```

Exercise 3: Update Inventory After Order

```
CREATE TRIGGER update_inventory
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
    UPDATE products
    SET stock = stock - NEW.quantity
```



```
WHERE id = NEW.product_id;  
END;
```

Exercise 4: Recalculate Final Score Before Insert

```
CREATE TRIGGER calc_final_score  
BEFORE INSERT ON exam_scores  
FOR EACH ROW  
BEGIN  
    SET NEW.final_score = NEW.assignment_score * 0.3 + NEW.exam_score *  
0.7;  
END;
```

Section 3: FAQ – Know More

Q1. Can a trigger modify the data being inserted or updated?

Yes, but only in **BEFORE INSERT** or **BEFORE UPDATE** triggers using **SET NEW.column_name**.

Q2. Are triggers executed for each row?

Yes. Every trigger defined with **FOR EACH ROW** executes once **per affected row**.



Q3. Can I create a trigger that runs after a SELECT query?

No. Triggers only apply to data **modification** statements: INSERT, UPDATE, DELETE.

Q4. Are there limitations with triggers?

- Complex triggers can reduce performance.
 - Triggers may be harder to debug and maintain.
 - Use only for tasks that must happen every time automatically.
-

Q5. Can I have multiple triggers on the same table?

Yes. As long as the trigger names are unique, you can have separate triggers for INSERT, UPDATE, and DELETE.

End of Notes for Chapter: Strategically Automating SQL